Лабораторная работа 5 Тестирование данных модели и pipeline

Методические материалы курса · учебный год 2026–2027

Версия: 2026–2027 учебный год Формат: команда 2–5 человек с индивидуальной сдачей Предшествующая работа: ЛР4 «Эксперименты и базовая оценка модели»

1 Цель

Построить проверяемую пирамиду тестов для сквозного ML-pipeline: отделить быстрые unit-тесты от контрактных и интеграционных проверок, подготовить fixtures корректных и испорченных данных, получить воспроизводимые pass/fail отчёты и доказать, что тест контролирует конкретный риск.

2 Результаты обучения

Каждый студент должен уметь:

- отличать тест функции, контракта, модели, API и интеграции;
- использовать pytest fixtures, tmp_path, параметризацию и pytest.raises;
- создавать минимальный набор данных для одного сценария;
- проверять отказ при неверной схеме, утечке, одном классе и некорректной вероятности;
- тестировать инварианты model bundle и форму предсказания;
- формировать JUnit XML и текстовый отчёт;
- объяснять, какой риск покрывает тест и почему тест может дать ложную уверенность;
- диагностировать намеренно внесённый дефект независимо от команды.

3 Командный результат

Команда предоставляет:

1. структуру тестов по слоям data, evaluation, model, API и pipeline;
2. общие fixtures в tests/conftest.py без зависимости от порядка тестов;
3. не менее трёх параметризованных негативных сценариев данных;
4. тест метрик с обоими классами и отказом при одном классе;
5. тест model bundle и детерминированного обучения;
6. интеграционный smoke test validate -> train -> load -> predict;
7. положительный отчёт reports/tests/lr5-pass.xml;
8. сохранённый фрагмент отрицательного отчёта с именем упавшего теста и причиной;
9. таблицу «риск — тест — ожидаемый отказ»;
10. issue, pull request, review и индивидуальные evidence cards;
11. tag lr5-v1 после успешной приёмки.

4 Пирамида тестов курса

Слой	Что проверяет	Пример	Частота
Unit	чистую функцию	вычисление metrics и validation правила	каждый commit
Contract	границу компонента	поля данных или API response	каждый pull request
Model	инварианты артефакта	признаки, версия, диапазон вероятности	каждый pull request
Integration	связку компонентов	validate, train, load, predict	каждый pull request
End-to-end	пользовательский путь	сервис в контейнере	перед release

ЛР5 концентрируется на первых четырёх слоях. Полный контейнерный end-to-end будет добавлен после ЛР6 и ЛР7.

5 Роли

Роль	Зона ответственности	Индивидуальное доказательство
Data test engineer	fixtures и негативные данные	параметризованный тест и объяснение правил
Model test engineer	bundle и предсказания	тест инварианта модели
Evaluation test engineer	metrics и пороги	тест обоих классов и исключения
Integration engineer	сквозной smoke test	отчёт и диагностика слоя отказа
Quality reviewer	структура, PR и test report	содержательное review и defect injection

Для команды 2–4 роли совмещаются. Один студент не остаётся единственным владельцем тестов более двух последовательных ЛР.

6 Подготовка

```
git switch main
git pull --ff-only
git switch -c lab5/testing-pipeline
python -m pip install -e ".[dev]"
python -m pytest --collect-only -q
```

Сначала сохраните исходный успешный отчёт. Если базовые тесты уже падают, ЛР5 начинается с отдельной issue по регрессии; нельзя скрывать исходный дефект добавлением skip.

7 Требования к тестам

Хороший тест имеет явное имя, минимальные входы, одну основную причину отказа и диагностическое сообщение. Он не зависит от сети, MLflow server, порядка выполнения и файлов, оставленных предыдущим тестом.

Запрещены проверки вида assert True, тесты без assertion, ловля любого исключения без проверки типа и тесты, которые изменяют эталонный CSV. Каждый временный artifact создаётся через tmp_path.

8 Общие fixtures

Создайте tests/conftest.py:

```
from pathlib import Path

import pandas as pd
import pytest


@pytest.fixture
```

```
def valid_frame() -> pd.DataFrame:
    return pd.read_csv(Path(__file__).parents[1] / "data" / "flights_sample.csv")


@pytest.fixture
def csv_factory(tmp_path: Path):
    def write(frame: pd.DataFrame, name: str = "input.csv") -> Path:
        path = tmp_path / name
        frame.to_csv(path, index=False)
        return path
    return write
```

Fixture возвращает новое состояние для теста. Не изменяйте объект session scope, если мутация может повлиять на другие сценарии.

9 Параметризованные тесты данных

Используйте @pytest.mark.parametrize для нескольких независимых нарушений:

```
@pytest.mark.parametrize(
    ("column", "value", "check_name"),
    [
        ("DepHour", 25, "DepHour.maximum"),
        ("IsWeekend", 2, "IsWeekend.allowed_values"),
        ("FlightDate", "bad-date", "FlightDate.date"),
    ],
)
def test_invalid_values_are_rejected(valid_frame, csv_factory, column, value, check_name):
    frame = valid_frame.copy()
    frame.loc[0, column] = value
    result = validate_dataset(csv_factory(frame), CONTRACT)
    assert result["passed"] is False
    assert any(check["name"] == check_name and not check["passed"] for check in result[
"checks"])
```

Идентификатор параметризованного case должен позволять понять вход без чтения кода. При необходимости задайте ids.

10 Тесты оценки

Функция метрик должна получать y_true, probabilities и threshold и проверять предусловия:

- длины совпадают;
- probabilities находятся в диапазоне от 0 до 1;
- threshold находится строго между 0 и 1;
- в y_true присутствуют оба класса.

Положительный тест проверяет ключи и диапазоны. Отрицательные используют pytest.raises(ValueError, match=...). Не закрепляйте слишком точное значение floating point без необходимости; используйте pytest.approx.

11 Тест model bundle

После обучения bundle обязан содержать:

- model с методом predict_proba;
- точный порядок features;
- версию формата;
- параметры обучения;
- границы split;
- validation-метрики.

Предсказание на корректной строке должно иметь форму (1, 2), а вероятность положительного класса — находиться в [0, 1]. Версия, которую возвращает API, должна совпадать с bundle.

12 Детерминированность

Обучите модель дважды с одинаковыми данными, параметрами и random_state. Сравните validation ROC AUC и probabilities на одном validation-фрагменте через pytest.approx.

Затем измените seed. Тест не обязан требовать другую метрику: на маленьком наборе результат может совпасть. Он должен проверять, что seed сохранён в bundle и отражён в run metadata.

13 Интеграционный smoke test

Минимальный путь:

1. проверить контракт;
2. обучить модель во временный файл;
3. загрузить bundle;
4. построить DataFrame в порядке features;
5. вызвать predict_proba;
6. проверить форму и диапазон;
7. не оставлять artifact вне tmp_path.

Интеграционный тест не заменяет unit-тесты: если он падает, unit-набор должен помочь локализовать слой.

14 Defect injection

Каждый участник временно вносит один контролируемый дефект в отдельной ветке или незакоммиченной копии:

- меняет порог цели >= 15 на > 15;
- меняет порядок FEATURES;
- удаляет проверку одного класса;
- разрешает DepHour=24;
- удаляет поле версии bundle.

Студент заранее называет тест, который должен упасть, запускает его по node ID и сохраняет фрагмент отчёта. После демонстрации дефект удаляется; в принимаемый tag он не входит.

15 Отчёты

Положительный отчёт:

```
python -m pytest -q --junitxml=reports/tests/lr5-pass.xml
```

Отрицательный отчёт сохраняется как небольшой текстовый фрагмент с командой, node ID, assertion и причиной. Не добавляйте огромный лог окружения и секреты.

Таблица рисков:

Риск	Тест	Ожидаемый сигнал
строка из будущего попала в train	chronological split test	assertion о границе времени
validation содержит один класс	evaluation precondition	ValueError
схема API и bundle расходятся	API contract test	неверная версия или 5xx
модельный файл повреждён	load smoke test	контролируемая ошибка загрузки

16 Индивидуальная сдача участника команды

Командный результат: tag lr5-v1, успешный JUnit XML и таблица рисков.

Поле	Что указать
Студент	ФИО, группа, команда, GitHub-логин
Роль	роль в ЛР5 и отличие от ЛР4
Задача	issue с критерием приёмки
Личный артефакт	test node ID, fixture, PR, review или отчёт
Контролируемый риск	ошибка, которую обнаруживает тест
Defect injection	внесённый дефект, ожидаемый и фактический failing test
Диагностика	первый полезный фрагмент traceback и причина
Воспроизведение	точная команда запуска одного теста и всего набора
Использование ИИ	инструмент и самостоятельно проверенные утверждения либо отсутствие использования

Минимальный допуск: tag или commit команды зафиксирован, личный test node ID существует, PR прошёл review другого участника, полный набор зелёный, отрицательный сценарий воспроизводится.

17 Защита

За 3–4 минуты участник:

1. запускает свой тест по node ID;
2. показывает риск и fixture;
3. временно активирует согласованный дефект;
4. объясняет assertion или исключение;
5. восстанавливает корректный код;
6. повторно получает успешный тест;
7. отвечает, почему соседний слой не заменяет эту проверку.

Преподаватель может назначить другой параметризованный case или попросить уменьшить fixture до минимального контрпримера.

18 Повторная сдача и стоп-факторы

Повторная сдача выполняется через новую issue или переоткрытую исходную, отдельный PR и обновлённую evidence card. История failed run сохраняется.

Стоп-факторы:

- тест всегда успешен или не содержит assertion;
- студент не может объяснить причину отказа;
- эталонные данные изменяются тестом;
- тест зависит от порядка или внешней сети без маркировки;
- дефект остаётся в принимаемой версии;
- отчёт сфальсифицирован или скопирован у другой команды.

19 Критерии оценивания

Критерий	Баллы	Полное выполнение
Fixtures и структура	2	изолированные fixtures и понятные слои тестов
Данные и контракт	2	параметризованные положительные и отрицательные cases
Модель и metrics	2	предусловия, bundle и диапазоны проверены
Интеграция и отчёты	1	smoke test и JUnit XML воспроизводимы
Командный процесс	1	issue, PR и содержательное review
Индивидуальная защита	2	defect injection, диагностика и объяснение риска

Максимум: 10 баллов. Рекомендуемый порог зачёта: 6 баллов при отсутствии стоп-факторов.

20 Чек-лист

- [ ] Команда состоит из 2–5 человек, роли ротированы.
- [ ] Тесты не зависят от порядка выполнения.
- [ ] Временные файлы создаются через tmp_path.
- [ ] Есть минимум три параметризованных нарушения данных.
- [ ] Метрики отказывают при одном классе и неверных probabilities.
- [ ] Bundle содержит обязательные поля и загружается.
- [ ] Интеграционный smoke test проходит.
- [ ] JUnit XML создан автоматически.
- [ ] Каждый участник выполнил defect injection.
- [ ] PR прошёл review и создан tag lr5-v1.

21 Связь с ЛР6

В ЛР6 эти проверки войдут в контейнерную сборку. Docker image не принимается, если unit, contract и integration tests не проходят до build.

22 Официальные источники

- pytest documentation: https://docs.pytest.org/en/stable/
- pytest parametrization: https://docs.pytest.org/en/stable/how-to/parametrize.html
- scikit-learn metrics: https://scikit-learn.org/stable/api/sklearn.metrics.html